

Capa de datos para un copiloto RAG interno de una distribuidora mayorista

Martín Madrid César Andrés Orellana Nicolás Valentín Ciarrapico

Trabajo Práctico Integrador – Bases de Datos para Inteligencia Artificial
Carrera de Especialización en Inteligencia Artificial – FIUBA
2026

Resumen

En este trabajo diseñamos la capa de datos de un copiloto RAG interno para una distribuidora mayorista. El problema que buscamos resolver es que la información relevante está repartida entre documentos, procedimientos, condiciones comerciales y datos operativos, por lo que encontrar una respuesta confiable puede ser lento y depender del conocimiento informal de las personas.

La solución propuesta usa PostgreSQL como motor principal, junto con las extensiones `pgvector` y `btree_gist`. En la misma base se guardan los datos operativos, los metadatos de los documentos, los fragmentos, los embeddings, los permisos y los eventos de auditoría. Elegimos este enfoque porque permite aplicar la vigencia de los documentos y los permisos antes de hacer la búsqueda por similitud. Así evitamos recuperar una fuente vencida o que el usuario no tiene permitido ver.

La implementación incluye datos sintéticos reproducibles, nueve consultas representativas, restricciones de integridad, control de acceso mediante RLS y evidencias de las respuestas generadas. El trabajo no implementa una interfaz, una API ni un modelo de lenguaje real: el objetivo es demostrar que la capa de datos puede sostener ese tipo de aplicación de manera consistente, trazable y con posibilidades de crecimiento.

Índice

1. Descripción del caso de uso	3
2. Relevamiento de datos necesarios	3
3. Clasificación de los datos según su tipo	4
4. Modelo conceptual	5
5. Modelo de implementación según la tecnología elegida	5
6. Decisiones de normalización, embebido, referencia y desnormalización	6
7. Justificación de la tecnología seleccionada	6
8. Implementación mínima realizada	6
9. Datos de ejemplo utilizados	6
10.Consultas representativas	6
11.Propuesta para datos semiestructurados, no estructurados y vectoriales	7
12.Propuesta de arquitectura de datos	7
13.Estrategia de seguridad, permisos y aislamiento	8
14.Consideraciones de escalabilidad y rendimiento	8
15.Conclusiones	9
A. Reproducibilidad y material de apoyo	9

1. Descripción del caso de uso

El caso elegido es el de un sistema RAG para consultar documentación técnica. Lo adaptamos al contexto de una distribuidora mayorista, ya que en este tipo de empresa la información está distribuida entre catálogos de productos, condiciones comerciales, pedidos, entregas, incidencias, procedimientos de logística y documentos de proveedores o de cumplimiento. En la práctica, una persona que necesita resolver una consulta debe buscar en varios documentos o preguntarle a alguien que conozca el proceso. Esto puede generar demoras, respuestas diferentes para una misma pregunta y uso de documentación que ya no está vigente.

Nuestra propuesta es diseñar la capa de datos que permitiría a un copiloto interno recuperar información relevante y usarla como fuente para responder. El foco no está en entrenar un modelo ni en crear una aplicación completa. El foco es organizar los datos para que una futura aplicación de IA pueda acceder a información correcta, vigente y autorizada.

Los usuarios principales que consideramos son **Operaciones y Logística**, que consulta fichas de producto, procedimientos logísticos e incidencias autorizadas; **Comercial y Compras**, que consulta condiciones comerciales, políticas de venta e información autorizada de proveedores; y **Administración y Calidad**, que consulta documentación de cumplimiento, documentación legal y la trazabilidad de las respuestas. Además, consideramos servicios internos de ingesta, consulta y administración documental. Estos servicios no representan usuarios finales y tienen permisos mínimos según la tarea que realizan.

El sistema debe poder registrar documentos con versiones, determinar qué versión está vigente, dividir los documentos en fragmentos, asociar embeddings a esos fragmentos y recuperar los más cercanos a una consulta. También debe resolver consultas estructuradas, por ejemplo sobre pedidos, entregas o condiciones comerciales. Para cada respuesta exitosa se guarda la evidencia que indica qué fuente fue utilizada.

Los riesgos más importantes son recuperar contenido no autorizado, usar una versión sustituida o revocada, responder sin poder justificar la fuente y perder la trazabilidad de una consulta anterior. Por este motivo, las tres decisiones centrales del diseño son conservar el historial de versiones, aplicar permisos antes de la recuperación y guardar evidencia y auditoría de las interacciones.

El alcance del trabajo se limita a la capa de datos. No incluimos frontend, backend, API, autenticación de usuarios, integración real con un LLM ni embeddings generados por un modelo comercial. Estos componentes se consideran posibles consumidores futuros de la base diseñada.

2. Relevamiento de datos necesarios

Para diseñar la solución separamos los datos según el papel que tienen dentro del problema. No todos los datos tienen el mismo ciclo de vida ni se consultan de la misma forma. Por ejemplo, un pedido se consulta con filtros y relaciones exactas, mientras que un procedimiento escrito se recupera por similitud semántica.

Los datos operativos son necesarios porque el copiloto no debería resolver por similitud preguntas que tienen una respuesta exacta en la base. Por ejemplo, la condición comercial vigente se obtiene cruzando cliente, producto y fecha; los pedidos con entregas demoradas se obtienen relacionando pedidos, entregas e incidencias. Guardar estos datos de forma estructurada permite aplicar restricciones, realizar agregaciones y evitar ambigüedades.

Para el corpus documental necesitamos registrar más información que el texto. Cada documento tiene una clase, un nivel de sensibilidad, una procedencia y una o varias versiones. Cada versión conserva el archivo original mediante su ruta relativa, nombre, tipo MIME, tamaño y checksum SHA-256. De esta manera, la base no guarda el archivo completo, pero puede verificar que el archivo al que hace referencia es el esperado.

Cada versión se divide en fragmentos. El fragmento es la unidad que se vectoriza y también

Grupo	Datos principales	Uso dentro de la solución
Operativos	Productos, categorías, clientes, segmentos, proveedores, pedidos, líneas de pedido, entregas e incidencias.	Responder consultas exactas del negocio y relacionar documentos con el contexto operativo.
Comerciales	Condiciones comerciales, precios, descuentos y períodos de vigencia.	Consultar qué condición corresponde a un cliente y producto en una fecha determinada.
Documentales	Documentos, versiones, clases documentales, sensibilidad, archivos y fragmentos.	Mantener el corpus, su procedencia, su vigencia y la unidad de recuperación.
Vectoriales	Modelos de embedding y vectores de cada fragmento.	Resolver búsquedas por similitud sobre documentación autorizada.
Acceso	Actores, perfiles autorizados y permisos por clase documental.	Limitar qué documentos puede recuperar cada perfil.
Interacción y auditoría	Consultas, respuestas, evidencias y eventos de auditoría.	Explicar respuestas anteriores y registrar acciones relevantes.

Cuadro 1: Datos relevados para la solución.

la unidad que luego se puede mostrar como evidencia. Esto permite indicar con precisión qué sección de qué versión fue utilizada para fundamentar una respuesta, en lugar de asociar la respuesta a un documento completo.

Los datos de acceso son importantes porque no todos los perfiles pueden ver las mismas clases documentales. El permiso se modela como una relación entre perfil y clase documental. La ausencia de un permiso significa denegación, por lo que no se necesitan listas de excepciones individuales. Finalmente, los datos de interacción y auditoría permiten reconstruir quién consultó, con qué perfil, qué resultado obtuvo y qué evidencia se utilizó, sin duplicar el contenido sensible de los documentos en el registro de auditoría.

Para la implementación usamos datos sintéticos. Esto evita trabajar con datos personales, comerciales o documentos internos reales y permite repetir la carga con los mismos conteos, archivos, vectores y resultados esperados.

3. Clasificación de los datos según su tipo

Tipo	Ejemplos en la solución
Estructurados	Productos, pedidos, permisos, consultas y eventos.
Semiestructurados	Metadatos variables controlados mediante JSONB.
No estructurados	Archivos Markdown de versiones documentales.
Vectoriales	Embeddings de fragmentos documentales.
Sensibles y auditables	Permisos, accesos, consultas y trazabilidad.

[Pendiente: Completar clasificación operacional/analítica y justificar persistencia.]

4. Modelo conceptual

La Figura 1 presenta las entidades, relaciones y cardinalidades del dominio. **[Pendiente:**

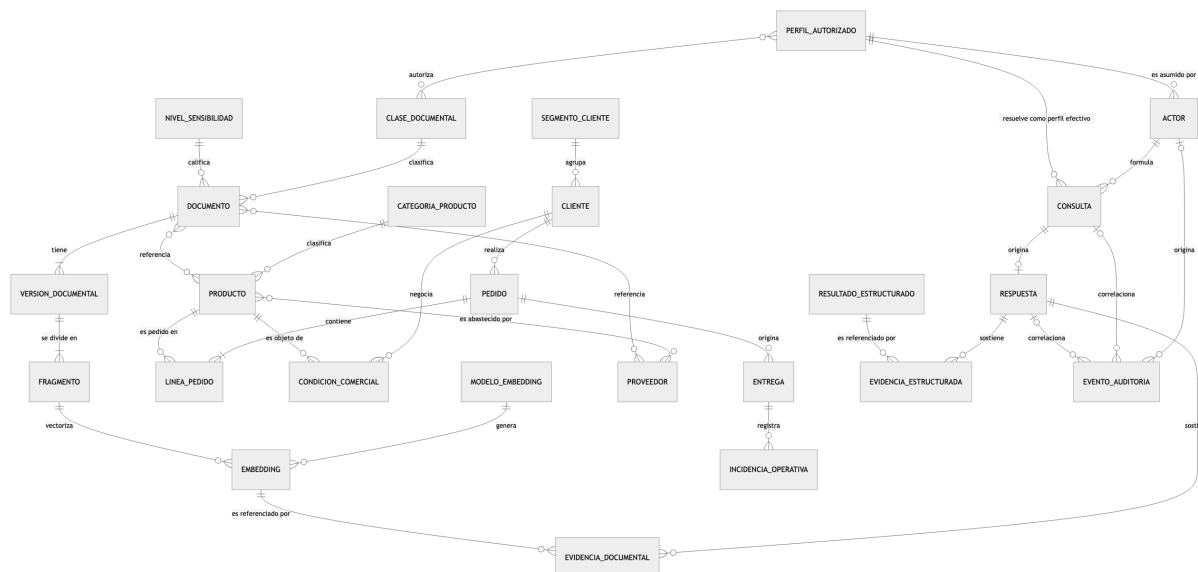


Figura 1: Modelo conceptual del dominio.

Interpretar relaciones relevantes y restricciones del dominio.]

5. Modelo de implementación según la tecnología elegida

El modelo lógico relacional y el físico se muestran en las Figuras 2 y 3; su fuente ejecutable es db/estructura/01_schema.sql. **[Pendiente: Describir tablas, claves, restricciones,**

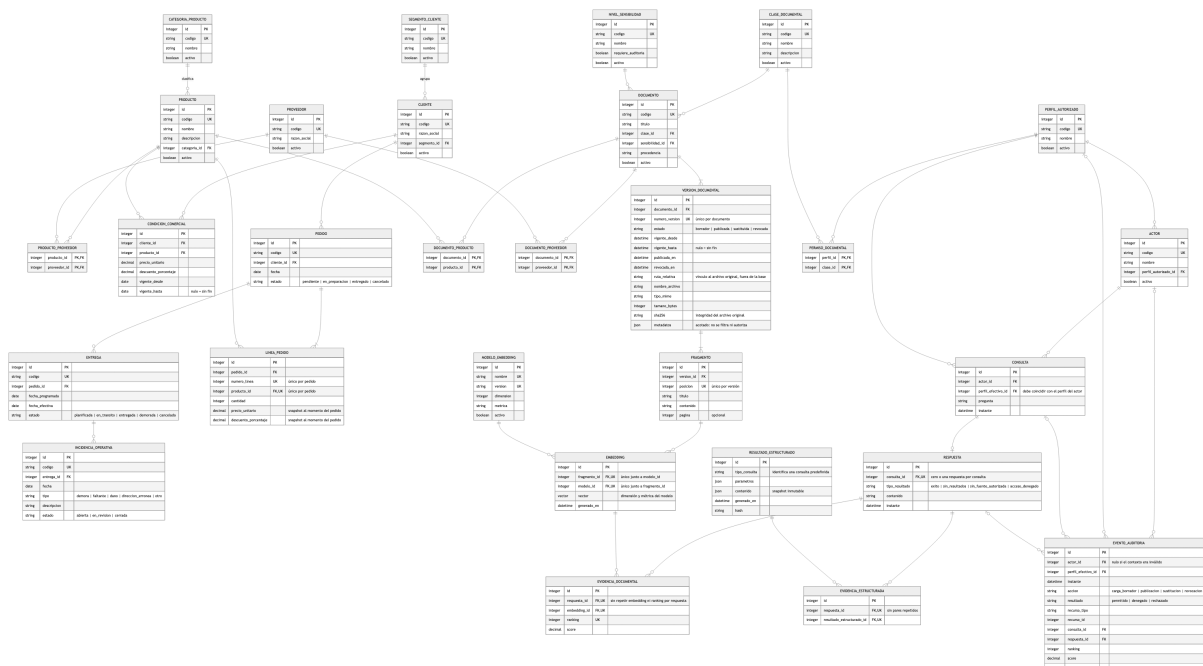


Figura 2: Modelo lógico relacional.

índices y el vínculo documento-versión-fragmento-embedding.]

vectorial; recuperación híbrida autorizada; matriz de autorización; y trazabilidad e inmutabilidad. **[Pendiente: Para cada consulta: pregunta, patrón SQL, resultado esperado y utilidad.]**

11. Propuesta para datos semiestructurados, no estructurados y vectoriales

Cada fragmento documental es la unidad vectorizada. Su embedding conserva la referencia a fragmento, versión, documento y modelo, junto con metadatos para aplicar vigencia y autorización antes del ranking. **[Pendiente: Explicar métrica coseno, top-k, riesgos de recuperación incorrecta, desactualizada o no autorizada, y sus mitigaciones.]**

12. Propuesta de arquitectura de datos

Para esta primera versión proponemos una arquitectura simple, con una única instancia de PostgreSQL como núcleo de la solución. En esa instancia se concentran los datos operativos, los metadatos documentales, los embeddings, los permisos, las consultas, las respuestas y la auditoría. PostgreSQL tiene las extensiones `pgvector`, para guardar y consultar embeddings, y `btree_gist`, para controlar períodos de vigencia sin solapamientos.

Elegimos un motor único porque el caso necesita combinar en una misma consulta relaciones estructuradas, vigencia de documentos, permisos y búsqueda por similitud. Si separáramos desde el inicio la base relacional y la base vectorial, tendríamos que replicar metadatos como permisos, clase documental y vigencia, y mantenerlos sincronizados. Para el alcance del TP, esa complejidad no se justifica.

El recorrido de los datos comienza con el script `scripts/generar_datos.py`, que genera datos sintéticos, documentos Markdown y un manifiesto con conteos, checksums y oráculos de consulta. Luego `scripts/cargar_base.sh` crea la base y ejecuta los scripts de esquema, datos, índices, vistas y seguridad. Los documentos se registran junto con sus versiones; cada versión conserva información de vigencia y una referencia verificable al archivo original. Después, cada versión documental se divide en fragmentos y los fragmentos se asocian a embeddings sintéticos de dimensión 32 y a un modelo de embedding.

Cuando se ejecuta una consulta de recuperación, se utiliza la vista de fragmentos recuperables. Esta vista considera la actividad del documento y del modelo, el estado publicado y la vigencia. Las políticas RLS agregan el filtro de permisos del perfil efectivo. Por lo tanto, el ranking vectorial se hace sobre un universo de información ya vigente y autorizado. Si se obtiene una respuesta exitosa, se registra la evidencia documental o estructurada utilizada y se generan eventos de auditoría para reconstruir la interacción.

Los archivos Markdown se mantienen fuera de la base de datos. La base guarda solamente la ruta relativa y los datos necesarios para verificar su integridad. Esta decisión simplifica el modelo físico de la prueba y evita guardar binarios en tablas. En una evolución del sistema, los archivos podrían trasladarse a almacenamiento de objetos, manteniendo el mismo contrato de checksum y procedencia.

No incorporamos un Data Warehouse, Data Lake o Lakehouse porque las consultas del TP son principalmente operativas y transaccionales. Tampoco se requiere un proceso analítico masivo ni se cuenta con fuentes heterogéneas de gran volumen. Si en el futuro se necesitaran indicadores históricos complejos, análisis de uso del copiloto o reportes de negocio a gran escala, sería razonable extraer los eventos y hechos necesarios hacia una capa analítica separada. Esa separación no es necesaria para demostrar el diseño actual.

`vector_cosine_ops`. El índice está justificado por el patrón de recuperación esperado cuando el corpus crece, aunque en la muestra el optimizador elija un *sequential scan*: recorrer 36 vectores es más barato que entrar al índice. La evidencia de este comportamiento está documentada en `evidencias/planes_ejecucion.md`.

También se definieron índices B-tree para los filtros y relaciones usadas por las consultas representativas: condiciones comerciales por cliente y producto, entregas e incidencias, relaciones de pedidos, filtros de vigencia documental y correlaciones de auditoría. No se agregaron índices para todas las columnas, porque cada índice mejora algunas lecturas pero agrega espacio y costo en las escrituras.

Una primera estrategia de escalabilidad sería aumentar los recursos de la instancia de PostgreSQL y revisar periódicamente los planes de ejecución. Si el tráfico de lectura creciera, se podrían agregar réplicas de sólo lectura para reportes o consultas que no requieran consistencia inmediata. Las operaciones de publicación, permisos y auditoría deberían mantenerse en la instancia principal para conservar la consistencia transaccional.

En una etapa posterior se podrían separar algunos componentes. Los archivos originales podrían ir a almacenamiento de objetos; los eventos de auditoría podrían particionarse o moverse a una solución orientada a escritura masiva; y una base vectorial dedicada podría evaluarse si la cantidad de embeddings y el throughput de búsquedas crecieran varios órdenes de magnitud. Sin embargo, esa última alternativa requeriría mantener sincronizados los permisos y la vigencia con el índice vectorial, por lo que agrega complejidad y riesgo de inconsistencia.

La decisión actual privilegia simplicidad, consistencia y trazabilidad. A la escala de la prueba, un único motor reduce el costo operativo y permite aplicar autorización y vigencia en la misma consulta que calcula la similitud. El compromiso es que PostgreSQL tiene un límite de escalamiento vertical y que, en volúmenes muy grandes, será necesario revisar el particionado, las réplicas y la conveniencia de especializar componentes.

15. Conclusiones

[Pendiente: Sintetizar solución, validaciones alcanzadas, límites y evoluciones futuras.]

A. Reproducibilidad y material de apoyo

La carga y validación se reproducen con las instrucciones del `README.md`. Los planes de ejecución, la matriz de autorización y las validaciones SQL se mantienen como evidencia complementaria, sin sobrecargar el informe. **[Pendiente: Agregar una tabla breve de evidencias si resulta útil.]**